🐳 Docker Storage Drivers & Copy-on-Write (CoW) Mechanism

◈ Introduction

Docker uses **storage drivers** to manage **container file systems**. These storage drivers enable Docker to use a **layered filesystem** model, allowing for efficient storage, sharing, and fast container startup.

◈ What is a Layered Filesystem?

Each Docker image consists of **multiple read-only layers** stacked on top of each other. When a container is created from an image:

- Docker adds a **writeable layer** on top of the image layers.
- All changes (file creation, updates, deletions) happen in this top writable layer.

✅ **Benefits:** Efficient disk usage, fast image builds, and reuse of common layers across containers.

◈ Copy-on-Write (CoW) Concept

Copy-on-Write (CoW) is a mechanism used by Docker to avoid duplicating files unless absolutely necessary.

◆ How CoW Works:

- When a container reads a file → it's accessed from the **read-only image layer**.
- When a container **modifies** a file:
  - That file is **copied** to the container's **writeable layer**.
  - The container then modifies **only** this copy.

📤 This avoids altering the base image and ensures multiple containers can use the same image layers without conflict.

◈ Real-World Analogy

Think of the image layers as a **template**. When you run a container:

- You get a **notebook with a transparent plastic cover (read-only)**.
- You can't write on it, but you can put a **writeable sheet (writeable layer)** on top.
- If you want to change anything, you put it on the top sheet only.

◈ Example

Let's see how this works practically:

▯ Step-by-Step Demo:

```
# Step 1: Create a container from Ubuntu
docker run -dit --name demo ubuntu

# Step 2: Inside the container, create a new file
docker exec -it demo bash
echo "Hello from Docker!" > /hello.txt
exit
```

✅ hello.txt is now created in the **writeable layer** of the demo container.

▯ Try Modifying a Base File:

```
# Step 3: Check if /etc/hostname exists
docker exec -it demo cat /etc/hostname

# Step 4: Try modifying it
docker exec -it demo bash
echo "new-host" > /etc/hostname
exit
```

👉 When /etc/hostname is modified:

- Docker **copies** it from the base layer to the container's writeable layer.
- Then applies the change.
- The original image layer remains **untouched**.

◈ Storage Drivers Supported by Docker

Driver	Description
overlay2	Default on most Linux distributions; supports CoW
aufs	Older Ubuntu systems; layered filesystem
btrfs	CoW-enabled, requires special setup
zfs	Enterprise-grade, powerful CoW support
vfs	No layering; used mainly for testing
windowsfilter	Used on Windows containers

You can check your current driver using:

docker info | grep Storage

◈ Summary

Concept	Description
Layered Filesystem	Docker images are composed of multiple read-only layers
Writeable Layer	Each running container has its own writeable layer
Copy-on-Write	Files are copied to the writeable layer only when modified
Storage Drivers	Manage how Docker implements CoW and layers

◈ Best Practices

✅ Use overlay2 driver (default in most cases) for performance and compatibility.
✅ Avoid unnecessary writes inside containers to keep layers thin and efficient.
✅ Build optimized images with minimal layers to reduce storage and speed up builds.

📓 Docker Storage & Copy-on-Write – Practical Assignments with Solutions

---

◈ Assignment 1: Check the Current Storage Driver

📝 **Task:** Find out which storage driver is used by your Docker engine.

✅ **Solution:**

docker info | grep -i "Storage Driver"

Expected Output:
E.g., Storage Driver: overlay2

◈ Assignment 2: Create a Container and Add a File

📝 **Task:**

- Create a new container from ubuntu

- Create a file /myfile.txt with some content

✅ **Solution:**

```
docker run -dit --name test1 ubuntu
docker exec test1 bash -c "echo 'Hello Docker' > /myfile.txt"
```

◈ Assignment 3: Commit the Container and Verify Layers

📝 **Task:**

- Commit the container as a new image
- Verify it has a new layer

✅ **Solution:**

```
docker commit test1 my-custom-image
docker history my-custom-image
```

Check if a new layer with echo command is added.

◈ Assignment 4: Modify a Base File and Observe CoW

📝 **Task:**

- Modify /etc/hostname file inside a container
- Prove that original image remains unchanged

✅ **Solution:**

```
docker run -dit --name test2 ubuntu
docker exec test2 bash -c "echo 'new-host' > /etc/hostname"
docker diff test2
```

Check docker diff shows that /etc/hostname was **C** (changed) — only in the container layer.

◈ Assignment 5: List Layers of a Docker Image

📝 **Task:** Show all layers of the ubuntu image.

✅ **Solution:**

```
docker history ubuntu
```

Displays each image layer and size.

◈ Assignment 6: Inspect Image and File System Diff

📝 Task:

- Create a container
- Add/remove a file
- Use docker diff to see filesystem changes

✅ Solution:

```
docker run -dit --name test3 ubuntu
docker exec test3 bash -c "touch /newfile && rm /etc/motd"
docker diff test3
```

Expected Output:

- A /newfile (Added)
- D /etc/motd (Deleted)

◈ Assignment 7: Create Two Containers from Same Image

📝 Task:

- Create two containers from ubuntu
- Modify same file in each
- Prove each has its own CoW copy

✅ Solution:

```
docker run -dit --name cont1 ubuntu
docker run -dit --name cont2 ubuntu

docker exec cont1 bash -c "echo 'One' > /shared.txt"
docker exec cont2 bash -c "echo 'Two' > /shared.txt"

docker exec cont1 cat /shared.txt   # Output: One
docker exec cont2 cat /shared.txt   # Output: Two
```

Shows that each container has **its own copy** of the file.

◈ Assignment 8: Mount a Volume and Check CoW Behavior

📝 Task:

- Run a container with volume mounted at /data

- Create files inside /data
- Check if these persist after container removal

✅ **Solution:**

```
docker run -dit --name voltest -v myvol:/data ubuntu
docker exec voltest bash -c "echo 'Volume Test' > /data/test.txt"
docker rm -f voltest
docker run --rm -v myvol:/data ubuntu cat /data/test.txt
```

Expected: Output is Volume Test, confirming persistent storage is outside CoW layers.

◈ Assignment 9: Use overlay2 and Explore Layer Directory (Linux)

📝 **Task:**

- Find actual layer paths if using overlay2 driver
- Inspect file changes

✅ **Solution:** (Linux only, root access)

```
docker info | grep -i "overlay2"

ls /var/lib/docker/overlay2/
```

Each subfolder is a **layer**, but navigating this is for **advanced study only**.

◈ Assignment 🔟: Observe Layer Reuse with Multiple Containers

📝 **Task:**

- Create 3 containers from ubuntu
- Without making changes, check their image layers

✅ **Solution:**

```
docker run -dit --name c1 ubuntu
docker run -dit --name c2 ubuntu
docker run -dit --name c3 ubuntu

docker inspect c1 | grep -i overlay
docker inspect c2 | grep -i overlay
```

All containers share the **same read-only layers**, only their **writeable layer is unique**.

📌 Bonus Tip: Clean Up Docker Environment

To remove all test containers and images:

```
docker rm -f $(docker ps -aq)
docker rmi -f $(docker images -q)
docker volume prune -f
```